14. JSON, Encoding, and Time

14. JSON, Encoding, and Time

Phase: Phase 2 — Core language depth
Estimated time: 1 day
Slug: `14-json-encoding-time`

Learning objectives

- Marshal and unmarshal JSON with struct tags
- Handle time.Time and timezones carefully
- Validate unknown fields when needed
- Avoid floating point for money — use integers

Prerequisites

Complete prior numbered lessons. Type every example yourself.

JSON

```
type Event struct {
    ID   string    `json:"id"`
    At   time.Time `json:"at"`
    Name string    `json:"name,omitempty"`
}

b, err := json.Marshal(ev)
err = json.Unmarshal(b, &ev)
```

Use `Decoder` for streams; `Unmarshal` for whole buffers.

Time

```
now := time.Now().UTC()
t, err := time.Parse(time.RFC3339, s)
```

Store UTC; convert for display. Durations use `time.Duration` (nanoseconds under the hood).

Encoding discipline

Be explicit about omitempty, pointers for null vs empty, and custom MarshalJSON only when needed.

Common mistakes (especially from Python instincts)

- Parsing local times without timezone
- Using float64 for currency in JSON
- Silent ignore of unknown fields in security-sensitive configs

Exercises

Work in `exercises/14-json-encoding-time/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/14-json-encoding-time/` until you have an honest failing attempt.

Self-check

1. What does `omitempty` do?
2. Why prefer UTC storage?
3. Decoder vs Unmarshal?

Further reading

- https://pkg.go.dev/encoding/json
- https://pkg.go.dev/time